Document number: P3046R0
Date: 2023-11-10
Project: Programming Language C++
Reference: ISO/IEC IS 14882:2020
Reply to: Jens Maurer
jens.maurer@gmx.net

Core Language Working Group "ready" Issues for the November, 2023 meeting

References in this document reflect the section and paragraph numbering of document WG21 N4958.

1038. Overload resolution of `&x.static_func`

Section: 12.3 [over.over] **Status:** ready **Submitter:** Mike Miller **Date:** 2010-03-02

The Standard is not clear whether the following example is well-formed or not:

```
struct S {
    static void f(int);
    static void f(double);
};
S s;
void (*pf)(int) = &s.f;
```

According to 7.6.1.5 [expr.ref] bullet 4.3, you do function overload resolution to determine whether `x.f` is a static or non-static member function. 7.6.2.2 [expr.unary.op] paragraph 6 says that you can only take the address of an overloaded function in a context that determines the overload to be chosen, and the initialization of a function pointer is such a context (12.3 [over.over] paragraph 1) . The problem is that 12.3 [over.over] is phrased in terms of “an overloaded function name,” and this is a member access expression, not a name.

There is variability among implementations as to whether this example is accepted; some accept it as written, some only if the `&` is omitted, and some reject it in both forms.

Additional note (October, 2010):

A related question concerns an example like

```
struct S {
    static void g(int*) {}
    static void g(long) {}
} s;

void foo() {
    (&s.g)(0L);
}
```

Because the address occurs in a call context and not in one of the contexts mentioned in 12.3 [over.over] paragraph 1, the call expression in `foo` is presumably ill-formed. Contrast this with the similar example

```
void g1(int*) {}
void g1(long) {}

void foo1() {
    (&g1)(0L);
}
```

This call presumably is well-formed because 12.2.2.2 [over.match.call] applies to “the address of a set of overloaded functions.” (This was clearer in the wording prior to the resolution of issue 704: “...in this context using `&F` behaves the same as using the name `F` by itself.”) It's not clear that there's any reason to treat these two cases differently.

This question also bears on the original question of this issue, since the original wording of 12.2.2.2 [over.match.call] also described the case of an ordinary member function call like `s.g(0L)` as involving the “name” of the function, even though the *postfix-expression* is a member access expression and not a “name.” Perhaps the reference to “name” in 12.3 [over.over] should be similarly understood as applying to member access expressions?

Additional notes (February, 2023)

This appears to be resolved, in part by P1787R6 (accepted November, 2020).

CWG 2023-06-12

The clarifications in P1787R6 did not address the core of this issue, so it is kept open. In order to avoid confusion, a wording change to clarify the treatment (regardless of direction) seems advisable. CWG felt that the first and second examples should be treated consistently, and expressed a mild preferences towards making those ill-formed. It was noted that the reference to *id-expression* in 12.3 [over.over] can be understood to refer to the *id-expression* of a class member access.

This issue is resolved by issue 2725.

**1698. Files ending in **

Section: 5.2 [lex.phases] **Status:** ready **Submitter:** David Krauss **Date:** 2013-06-10

The description of how to handle file not ending in a newline in 5.2 [lex.phases] paragraph 1, phase 2, is:

2. Each instance of a backslash character (\) immediately followed by a new-line character is deleted, splicing physical source lines to form logical source lines. Only the last backslash on any physical source line shall be eligible for being part of such a splice. If, as a result, a character sequence that matches the syntax of a universal-character-name is produced, the behavior is undefined. A source file that is not empty and that does not end in a new-line character, or that ends in a new-line character immediately preceded by a backslash character before any such splicing takes place, shall be processed as if an additional new-line character were appended to the file.

This is not clear regarding what happens if the last character in the file is a backslash. In such a case, presumably the result of adding the newline should not be a line splice but rather a backslash *preprocessing-token* (that will be diagnosed as an invalid token in phase 7), but that should be spelled out.

CWG 2023-07-14

Addressed by the resolution for issue 2747.

2054. Missing description of class SFINAE

Section: 13.10.3 [temp.deduct] **Status:** ready **Submitter:** Ville Voutilainen **Date:** 2014-12-07

Presumably something like the following should be well-formed, where a deduction failure in a partial specialization is handled as a SFINAE case as it is with function templates and not a hard error:

```
template <class T, class U> struct X   {
  typedef char member;
};

template<class T> struct X<T,
 typename enable_if<(sizeof(T)>sizeof(
   float)), float>::type>
{
  typedef long long member;
};

int main() {
  cout << sizeof(X<double, float>::member);
}
```

However, this does not appear to be described anywhere in the Standard.

Additional notes (January, 2023)

The section on SFINAE (13.10.3.1 [temp.deduct.general] paragraph 8) is not specific to function templates, and 13.7.6.2 [temp.spec.partial.match] paragraph 2 hands off the "matching" determination for partial specializations to 13.10.3 [temp.deduct] in general. However, the definition of deduction substitution loci in 13.10.3.1 [temp.deduct.general] paragraph 7 does not account for the template argument list of a partial specialization.

Proposed resolution (approved by CWG 2023-11-08):

Change in 13.10.3.1 [temp.deduct.general] paragraph 7 as follows:

> The deduction substitution loci are
>
> - the function type outside of the *noexcept-specifier*,
> - the *explicit-specifier*, ~~and~~
> - the template parameter declarations~~.~~**, and**
> - **the template argument list of a partial specialization (13.7.6.1 [temp.spec.partial.general]).**
>
> The substitution occurs in all types and expressions that are used in the deduction substitution loci. ...

2102. Constructor checking in *new-expression*

Section: 7.6.2.8 [expr.new] **Status:** ready **Submitter:** Richard Smith **Date:** 2015-03-16

According to 7.6.2.8 [expr.new] paragraph 25,

> If the *new-expression* creates an object or an array of objects of class type, access and ambiguity control are done for the allocation function, the deallocation function (11.4.11 [class.free]), and the constructor (11.4.5 [class.ctor]).

The mention of “the constructor” here is strange. For the “object of class type” case, access and ambiguity control are done when we perform initialization in paragraph 17, and we might not be calling a constructor anyway (for aggregate initialization). This seems wrong.

For the “array of objects of class type” case, it makes slightly more sense (we need to check the trailing array elements can be default-initialized) but again (a) we aren't necessarily using a constructor, (b) we should say *which* constructor — and we may need overload resolution to find it, and (c) shouldn't this be part of initialization, so we can distinguish between the cases where we should copy-initialize from `{}` and the cases where we should default-initialize?

Additional notes (May, 2023):

It is unclear whether default-initialization is required to be well-formed even for an array with no elements.

Proposed resolution (approved by CWG 2023-06-16):

1. Insert a new paragraph before 7.6.2.8 [expr.new] paragraph 9:

> If the allocated type is an array, the *new-initializer* is a *braced-init-list*, and the *expression* is potentially-evaluated and not a core constant expression, the semantic constraints of copy-initializing a hypothetical element of the array from an empty initializer list are checked (9.4.5 [dcl.init.list]). [Note: The array can contain more elements than there are elements in the *braced-init-list*, requiring initialization of the remainder of the array elements from an empty initializer list. -- end note]
>
> Objects created by a new-expression have dynamic storage duration (6.7.5.5 [basic.stc.dynamic]). ...

2. Change in 7.6.2.8 [expr.new] paragraph 25 as follows:

> ~~If the *new-expression* creates an object or an array of objects of class type, access and ambiguity control are done for the allocation function, the deallocation function (6.7.5.5.3 [basic.stc.dynamic.deallocation]), and the constructor (11.4.5 [class.ctor]) selected for the initialization (if any).~~ If the *new-expression* creates an array of objects of class type, the destructor is potentially invoked (11.4.7 [class.dtor]).

3. Change in 7.6.2.8 [expr.new] paragraph 28 as follows:

> A declaration of a placement deallocation function matches the declaration of a placement allocation function if it has the same number of parameters and, after parameter transformations (9.3.4.6 [dcl.fct]), all parameter types except the first are identical. If the lookup finds a single matching deallocation function, that function will be called; otherwise, no deallocation function will be called. If the lookup finds a usual deallocation function and that function, considered as a placement deallocation function, would have been selected as a match for the allocation function, the program is ill-formed. For a non-placement allocation function, the normal deallocation function lookup is used to find the matching deallocation function (7.6.2.9 [expr.delete]). **In any case, the matching deallocation function (if any) shall be non-deleted and accessible from the point where the *new-expression* appears.**

4. Change in 9.4.1 [dcl.init.general] paragraph 7 as follows:

> To *default-initialize* an object of type T means:
> - ...
> - If T is an array type, **the semantic constraints of default-initializing a hypothetical element shall be met and** each element is default-initialized.
> - ...

5. Change in 9.4.1 [dcl.init.general] paragraph 9 as follows:

> To *value-initialize* an object of type T means:
> - ~~if~~ **If** T is a (possibly cv-qualified) class type (Clause 11 [class]), then
>   - if T has either no default constructor (11.4.5.2 [class.default.ctor]) or a default constructor that is user-provided or deleted, then the object is default-initialized;
>   - otherwise, the object is zero-initialized and the semantic constraints for default-initialization are checked, and if T has a non-trivial default constructor, the object is default-initialized~~;~~**.**
> - ~~if~~ **If** T is an array type, **the semantic constraints of value-initializing a hypothetical element shall be met and** each element is value-initialized~~;~~**.**
> - ~~otherwise~~**Otherwise**, the object is zero-initialized.

2252. Enumeration list-initialization from the same type

Section: 9.4.5 [dcl.init.list] **Status:** ready **Submitter:** Richard Smith **Date:** 2016-03-22

According to 9.4.5 [dcl.init.list] bullet 3.8,

> Otherwise, if `T` is an enumeration with a fixed underlying type (9.7.1 [dcl.enum]), the *initializer-list* has a single element `v`, and the initialization is direct-list-initialization, the object is initialized with the value `T(v)` (7.6.1.4 [expr.type.conv]); if a narrowing conversion is required to convert `v` to the underlying type of `T` , the program is ill-formed.

This could be read as requiring that there be a conversion from `v` to the underlying type of `T`, leaving the status of an example like the following unclear:

```
enum class E {};
struct X { operator E(); };
E{X()}; // ok?
```

Notes from the March, 2018 meeting:

CWG disagreed that the existing wording requires such a conversion, only that if such a conversion is possble, it must not narrow. A formulation along the lines of “if that initialization involves a narrowing conversion to the underlying type of `T`...” was suggested to clarify the intent. This will be handled editorially, and the issue will be left in "review" status until the change has been verified.

Additional notes (August, 2023)

Issue 2374 has meanwhile clarified that `v` is required to implicitly convert to the underlying type of the enumeration for 9.4.5 [dcl.init.list] bullet 3.8 to apply. Now, the logic falls through to 9.4.5 [dcl.init.list] bullet 3.9 for the above example, making it well-formed.

CWG 2023-09-15

Class types with conversions to scalar types were not in view when the wording in this bullet was conceived.

Proposed resolution (approved by CWG 2023-10-06):

Change in 9.4.5 [dcl.init.list] bullet 3.8 as follows:

> Otherwise, if T is an enumeration with a fixed underlying type (9.7.1 [dcl.enum]) U, the initializer-list has a single element v **of scalar type**, v can be implicitly converted to U, and the initialization is direct-list-initialization, the object is initialized with the value T(v) (7.6.1.4 [expr.type.conv]); if a narrowing conversion is required to convert v to U, the program is ill-formed.

2504. Inheriting constructors from virtual base classes

Section: 11.9.4 [class.inhctor.init] **Status:** ready **Submitter:** Hubert Tong **Date:** 2021-11-03

According to 11.9.4 [class.inhctor.init] paragraph 1,

> When a constructor for type `B` is invoked to initialize an object of a different type `D` (that is, when the constructor was inherited (9.9 [namespace.udecl])), initialization proceeds as if a defaulted default constructor were used to initialize the `D` object and each base class subobject from which the constructor was inherited, except that the `B` subobject is initialized by the invocation of the inherited constructor. The complete initialization is considered to be a single function call; in particular, the initialization of the inherited constructor's parameters is sequenced before the initialization of any part of the `D`object.

First, this assumes that the base class constructor will be invoked from the derived class constructor, which will not be true if the base is virtual and initialized by a more-derived constructor.

If the call to the virtual base constructor is omitted, the last sentence is unclear whether the initialization of the base class constructor's parameters by the inheriting constructor occurs or not. There is implementation divergence in the initialization of `V`'s parameter in the following example:

```
struct NonTriv {
  NonTriv(int);
  ~NonTriv();
};
struct V { V() = default; V(NonTriv); };
struct Q { Q(); };
struct A : virtual V, Q {
  using V::V;
  A() : A(42) { }
};
struct B : A { };
void foo() { B b; }
```

CWG telecon 2022-09-23:

Inheriting constructors from a virtual base class ought to be ill-formed. Inform EWG accordingly.

Possible resolution [SUPERSEDED]:

1. Change in 9.9 [namespace.udecl] paragraph 3 as follows:

   > ... If a *using-declarator* names a constructor, its *nested-name-specifier* shall name a direct **non-virtual** base class of the current class. If the immediate (class) scope is associated with a class template, it shall derive from the specified base class or have at least one dependent base class.

2. Change the example in 11.9.4 [class.inhctor.init] paragraph 1 as follows:

   `D2 f(1.0);` *// error:* `B1` *has ~~a deleted~~* ***no*** *default constructor*

   ~~`struct W { W(int); };`~~
   ~~`struct X : virtual W { using W::W; X() = delete; };`~~
   ~~`struct Y : X { using X::X; };`~~
   ~~`struct Z : Y, virtual W { using Y::Y; };`~~
   ~~`Z z(0);` *// OK, initialization of Y does not invoke default constructor of X*~~

3. Change the example in 11.9.4 [class.inhctor.init] paragraph 2 as follows:

   ~~`struct V1 : virtual B { using B::B; };`~~
   ~~`struct V2 : virtual B { using B::B; };`~~

   ~~`struct D2 : V1, V2 {`~~
   ~~`  using V1::V1;`~~
   ~~`  using V2::V2;`~~
   ~~`};`~~
   `D1 d1(0);` *// error: ambiguous*
   ~~`D2 d2(0);` *// OK, initializes virtual B base class, which initializes the A base class*~~
   ~~*// then initializes the V1 and V2 base classes as if by a defaulted default constructor*~~

CWG telecon 2022-10-07:

Given that there are examples that discuss inheriting constructors from virtual base classes and given the existing normative wording, making it clear that `NonTriv` is not constructed, CWG felt that the implementation divergence is best addressed by amending the examples.

Possible resolution [SUPERSEDED]:

Add another example before 11.9.4 [class.inhctor.init] paragraph 2 as follows:

> [Example:

```
struct NonTriv {
  NonTriv(int);
  ~NonTriv();
};
struct V { V() = default; V(NonTriv); };
struct Q { Q(); };
struct A : virtual V, Q {
  using V::V;
  A() : A(42) { }     // #1, A(42) is equivalent to V(42)
};
struct B : A { };
void foo() { B b; }
```

In this example, the `V` subobject of `b` is constructed using the defaulted default constructor. The *mem-initializer* naming the constructor inherited from `V` at #1 is not evaluated and thus no object of type `NonTriv` is constructed. -- end example]

If the constructor was inherited from multiple base class subobjects of type B, the program is ill-formed.

Proposed resolution (approved by CWG 2023-11-06):

1. Change in 11.9.4 [class.inhctor.init] paragraph 1 as follows:

> When a constructor for type `B` is invoked to initialize an object of a different type `D` (that is, when the constructor was inherited (9.9 [namespace.udecl])), initialization proceeds as if a defaulted default constructor were used to initialize the `D` object and each base class subobject from which the constructor was inherited, except that the `B` subobject is initialized by ~~the invocation of~~ the inherited constructor **if the base class subobject were to be initialized as part of the `D` object (11.9.3 [class.base.init])**. **The invocation of the inherited constructor, including the evaluation of any arguments, is omitted if the `B` subobject is not to be initialized as part of the `D` object.** The complete initialization is considered to be a single function call; in particular, **unless omitted,** the initialization of the inherited constructor's parameters is sequenced before the initialization of any part of the `D`object.

2. Add another example before 11.9.4 [class.inhctor.init] paragraph 2 as follows:

[Example:

```
struct V { V() = default; V(int); };
struct Q { Q(); };
struct A : virtual V, Q {
  using V::V;
  A() = delete;
};
int bar() { return 42; }
struct B : A {
  B() : A(bar()) {}  // ok
};
struct C : B {};
void foo() { C c; } // bar is not invoked, because the V subobject is not initialized as part of B
```

-- end example]

CWG telecon 2022-10-21:

This is an ABI break for implementations when transitioning to the C++17 model for inheriting constructors.

2531. Static data members redeclared as constexpr

Section: 9.2.6 [dcl.constexpr] **Status:** ready **Submitter:** Davis Herring **Date:** 2022-02-16

C++17 made constexpr static data members implicitly inline (9.2.6 [dcl.constexpr] paragraph 1):

> A function or static data member declared with the `constexpr` or `consteval` specifier is implicitly an inline function or variable (9.2.8 [dcl.inline]).

However, that makes the following well-formed C++14 program ill-formed, no diagnostic required, per 9.2.8 [dcl.inline] paragraph 5:

> If a function or variable with external or module linkage is declared inline in one definition domain, an inline declaration of it shall be reachable from the end of every definition domain in which it is declared; no diagnostic is required.

```
// x.hh
struct X {
  static const int x;
};

// TU 1
#include "x.hh"
constexpr int X::x{};

// TU 2
#include "x.hh"
int main() { return !&X::x; }
```

Proposed resolution (reviewed by CWG 2023-02-07, approved by CWG 2023-11-07):

Change 9.2.6 [dcl.constexpr] paragraph 1 as follows:

> A function or static data member declared with the `constexpr` or `consteval` specifier **on its first declaration** is implicitly an inline function or variable (9.2.8 [dcl.inline]).

Drafting note: Functions must be declared `constexpr` on every declaration if on any, so this isn't a change for them.

2556. Unusable `promise::return_void`

Section: 8.7.5 [stmt.return.coroutine] **Status:** ready **Submitter:** Davis Herring **Date:** 2022-03-24

Subclause 8.7.5 [stmt.return.coroutine] paragraph 3 specifies:

> If `p.return_void()` is a valid expression, flowing off the end of a coroutine's *function-body* is equivalent to a `co_return` with no operand; otherwise flowing off the end of a coroutine's *function-body* results in undefined behavior.

However, 9.5.4 [dcl.fct.def.coroutine] paragraph 6 suggests:

> If searches for the names `return_void` and `return_value` in the scope of the promise type each find any declarations, the program is ill-formed. [*Note*: If `return_void` is found, flowing off the end of a coroutine is equivalent to a `co_return` with no operand. Otherwise, flowing off the end of a coroutine results in undefined behavior (8.7.5 [stmt.return.coroutine]). —*end note*]

The difference is between the conditions "valid expression" and "found by name lookup". Effectively, it means that undefined behavior might result where the implementation could instead diagnose an ill-formed use of `return_void` (for example, because it is inaccessible, deleted, or the function call requires arguments).

Proposed resolution (approved by CWG 2023-06-17):

Change in 8.7.5 [stmt.return.coroutine] paragraph 3 as follows:

> If ~~`p.return_void()` is a valid expression~~ **a search for the name `return_void` in the scope of the promise type finds any declarations**, flowing off the end of a coroutine's *function-body* is equivalent to a `co_return` with no operand; otherwise flowing off the end of a coroutine's *function-body* results in undefined behavior.

2570. Clarify constexpr for defaulted functions

Section: 9.5.2 [dcl.fct.def.default] **Status:** ready **Submitter:** Gabriel dos Reis **Date:** 2022-04-18

After the application of P2448R2, 9.5.2 [dcl.fct.def.default] paragraph 3 reads:

> A function explicitly defaulted on its first declaration is implicitly inline (9.2.8 [dcl.inline]), and is implicitly constexpr (9.2.6 [dcl.constexpr]) if it satisfies the requirements for a constexpr function.

It is unclear that no other such defaulted function is implicitly constexpr.

Proposed resolution (approved by CWG 2023-06-17):

> A function explicitly defaulted on its first declaration is implicitly inline (9.2.8 [dcl.inline]), and is implicitly constexpr (9.2.6 [dcl.constexpr]) if it satisfies the requirements for a constexpr function. **[*Note:* Other defaulted functions are not implicitly constexpr. -- *end note*]**

2591. Implicit change of active union member for anonymous union in union

Section: 11.5.1 [class.union.general] **Status:** ready **Submitter:** Richard Smith **Date:** 2022-05-29

Subclause 11.5.1 [class.union.general] paragraph 6 describes how union member subobjects are implicitly created by certain assignment operations that assign to union members. However, this description does not appear to properly handle the case of an anonymous union appearing within a union:

```
union A {
  int x;
  union {
   int y;
  };
};
void f() {
  A a = {.x = 1};
  a.y = 2;
}
```

Here, the expectation is that the assignment to `a.y` starts the lifetime of the anonymous union member subobject within `A` and also the int member subobject of the anonymous union member subobject. But the algorithm for computing S(`a.y`) determines that it is `{a.y}` and does not include the anonymous union member subobject.

Proposed resolution (approved by CWG 2023-06-17):

Change in 11.5.1 [class.union.general] paragraph 6 as follows:

In an assignment expression of the form `E1 = E2` that uses either the built-in assignment operator (7.6.19 [expr.ass]) or a trivial assignment operator (11.4.6 [class.copy.assign]), for each element X of S(E1) **and each anonymous union member X (11.5.2 [class.union.anon]) that is a member of a union and has such an element as an immediate subobject (recursively)**, if modification of X would have undefined behavior under 6.7.3 [basic.life], an object of the type of X is implicitly created in the nominated storage; no initialization is performed and the beginning of its lifetime is sequenced after the value computation of the left and right operands and before the assignment.

Editing note: Adding this rule into the definition of S would be more logical, but S(E) is a set of subexpressions of E and there is no form of expression that names an anonymous union member. Redefining S(E) to be a set of objects might be a better option.

2595. "More constrained" for eligible special member functions

Section: 11.4.4 [special] **Status:** ready **Submitter:** Barry Revzin **Date:** 2022-06-08

Consider:

```
#include <type_traits>

template<typename T>
concept Int = std::is_same_v<T, int>;

template<typename T>
concept Float = std::is_same_v<T, float>;

template<typename T>
struct Foo {
  Foo() requires Int<T> = default; // #1
  Foo() requires Int<T> || Float<T> = default; // #2
};
```

Per the wording, #1 is not eligible for `Foo<float>`, because the constraints are not satisfied. But #2 also is not eligible, because #1 is more constrained than #2. The intent is that #2 is eligible.

Proposed resolution (approved by CWG 2023-06-17):

Change in 11.4.4 [special] paragraph 6 as follows:

> An *eligible special member function* is a special member function for which:
>
> - the function is not deleted,
> - the associated constraints (13.5 [temp.constr]), if any, are satisfied, and
> - no special member function of the same kind **whose associated constraints, if any, are satisfied** is more constrained (13.5.5 [temp.constr.order]).

2600. Type dependency of placeholder types

Section: 13.8.3.3 [temp.dep.expr] **Status:** ready **Submitter:** Hubert Tong **Date:** 2022-06-18

Subclause 13.8.3.2 [temp.dep.type] paragraph 7 has a list of types considered to be dependent. This list covers placeholder types only insofar as it has an entry about `decltype(expression)`. Subclause 13.8.3.3 [temp.dep.expr] paragraph 3 has a list of expression forms not considered dependent unless specific types named by the expressions are dependent. This list includes forms where placeholder types are allowed. For example, the wording does not say that the *new-expression* at #1 (below) is dependent, but it ought to be:

```
template <typename T> struct A { A(bool, T); };

void g(...);

template <typename T>
auto f(T t) { return g(new A(t, 0)); }  // #1

int g(A<int> *);
int h() { return f<void *>(nullptr); }
```

Some implementation even treats an obviously non-dependent case as dependent:

```
template <typename T, typename U> struct A { A(T, U); };

void g(...); // #1

template <typename T>
auto f() { return g(new A(0, 0)); } // #1 or #2?

int g(A<int, int> *); // #2
void h() { return f<void *>(); }
```

A similar example that is non-dependent:

```
template <typename T, typename U = T> struct A { A(T, U); };

void g(...);

template <typename T>
```

```
auto f() { return g(new A(0, 0)); }

int g(A<int> *);
void h() { return f<void *>(); }
```

And another non-dependent one:

```
template <typename T, typename U = T> struct A { A(T); };

void g(...);

template <typename T>
auto f() { return g(new A(0)); }

int g(A<int> *);
void h() { return f<void *>(); }
```

And here is an example that is dependent:

```
template<class T>
struct S {
 template<class U = T> struct A { A(int); };

 auto f() { return new A(0); } // dependent return type
};
```

Proposed resolution (November, 2022) [SUPERSEDED]:

1. Change in 7.6.2.8 [expr.new] paragraph 2 as follows:

   If a placeholder type (9.2.9.6 [dcl.spec.auto]) **or a placeholder for a deduced class type (9.2.9.7 [dcl.type.class.deduct])** appears in the *type-specifier-seq* of a *new-type-id* or *type-id* of a *new-expression*, the allocated type is deduced as follows: Let init be the *new-initializer* , if any, and T be the *new-type-id* or *type-id* of the *new-expression*, then the allocated type is the type deduced for the variable x in the invented declaration (9.2.9.6 [dcl.spec.auto]):

   ```
   T x init ;
   ```

2. Insert new paragraphs before 13.8.3.2 [temp.dep.type] paragraph 7 and change the latter as follows:

   An initializer is dependent if any constituent expression (6.9.1 [intro.execution]) of the initializer is type-dependent. A placeholder type (9.2.9.6.1 [dcl.spec.auto.general]) is dependent if it designates a type deduced from a dependent initializer.

   A placeholder for a deduced class type (9.2.9.7 [dcl.type.class.deduct]) is dependent if

   - **it has a dependent initializer or**
   - **any default *template-argument* of the primary class template named by the placeholder is dependent when considered in the scope enclosing the primary class template.**

   A type is dependent if it is

   - ...
   - a function type whose exception specification is value-dependent,
   - **denoted by a dependent placeholder type,**
   - **denoted by a dependent placeholder for a deduced class type,**
   - ...

Proposed resolution (approved by CWG 2023-06-12):

1. Change in 7.6.2.8 [expr.new] paragraph 2 as follows:

   If a placeholder type (9.2.9.6 [dcl.spec.auto]) **or a placeholder for a deduced class type (9.2.9.7 [dcl.type.class.deduct])** appears in the *type-specifier-seq* of a *new-type-id* or *type-id* of a *new-expression*, the allocated type is deduced as follows: Let init be the *new-initializer* , if any, and T be the *new-type-id* or *type-id* of the *new-expression*, then the allocated type is the type deduced for the variable x in the invented declaration (9.2.9.6 [dcl.spec.auto]):

   ```
   T x init ;
   ```

2. Insert new paragraphs before 13.8.3.2 [temp.dep.type] paragraph 7 and change the latter as follows:

   An initializer is dependent if any constituent expression (6.9.1 [intro.execution]) of the initializer is type-dependent. A placeholder type (9.2.9.6.1 [dcl.spec.auto.general]) is dependent if it designates a type deduced from a dependent initializer.

   A placeholder for a deduced class type (9.2.9.7 [dcl.type.class.deduct]) is dependent if

   - **it has a dependent initializer, or**
   - **it refers to an alias template that is a member of the current instantiation and whose *defining-type-id* is dependent after class template argument deduction (12.2.2.9 [over.match.class.deduct]) and substitution (13.7.8 [temp.alias]).**

   [Example:

   ```
   template<class T, class V>
   struct S { S(T); };

   template<class U>
   struct A {
     template<class T> using X = S<T, U>;
     template<class T> using Y = S<T, int>;
     void f() {
       new X(1);    // dependent
   ```

```
      new Y(1);    // not dependent
    }
  };
```

-- end example]

A type is dependent if it is

- ...
- a function type whose exception specification is value-dependent,
- **denoted by a dependent placeholder type,**
- **denoted by a dependent placeholder for a deduced class type,**
- ...

2628. Implicit deduction guides should propagate constraints

Section: 12.2.2.9 [over.match.class.deduct] **Status:** ready **Submitter:** Roy Jacobson **Date:** 2022-09-11

Consider:

```
template<class T> concept True = true;

template<class T> struct X {
  template<class U> requires True<T> X(T, U(&)[3]);
};
template<typename T, typename U> X(T, U(&)[3]) -> X<T>;
int arr3[3];
X z(3, arr3);     // #1
```

According to 12.2.2.9 [over.match.class.deduct] bullet 1.1, the *requires-clause* of the constructor is not propagated to the function template synthesized for the implicit deduction guide. Thus, instead of favoring the more-constrained implicit deduction guide per 12.2.4.1 [over.match.best.general] bullet 2.6, the user-declared *deduction-guide* is preferred per 12.2.4.1 [over.match.best.general] bullet 2.11.

Proposed resolution (approved by CWG 2023-10-20):

Change in 12.2.2.9 [over.match.class.deduct] bullet 1.1 as follows:

- If `C` is defined, for each constructor of `C`, a function template with the following properties:
  - The template parameters are the template parameters of `C` followed by the template parameters (including default template arguments) of the constructor, if any.
  - **The associated constraints (13.5.3 [temp.constr.decl]) are the conjunction of the associated constraints of `C` and the associated constraints of the constructor.**
  - The types of the function parameters are those of the constructor.
  - The return type is the class template specialization designated by `C` and template arguments corresponding to the template parameters of `C`.

2672. Lambda body SFINAE is still required, contrary to intent and note

Section: 13.10.3.1 [temp.deduct.general] **Status:** ready **Submitter:** Richard Smith **Date:** 2022-09-30

Subclause 13.10.3.1 [temp.deduct.general] paragraph 9 specifies:

> A *lambda-expression* appearing in a function type or a template parameter is not considered part of the immediate context for the purposes of template argument deduction. [*Note 7:* The intent is to avoid requiring implementations to deal with substitution failure involving arbitrary statements. ... -- end note]

However, the intent of the note is not satisfied by the normative rule, because a *lambda-expression* appearing in a *requires-expression* has the same concerns as one in a function signature.

Suggested resolution: Change the rule to say that substitution into the body of a lambda is never in the immediate context of substitution into the lambda-expression and move the rule somewhere more general.

Possible resolution (reviewed by CWG 2023-08-25) [SUPERSEDED]:

1. Change in 13.5.2.3 [temp.constr.atomic] paragraph 3 as follows:

> To determine if an atomic constraint is satisfied, the parameter mapping and template arguments are first substituted into its expression. If substitution results in an invalid type or expression **in the immediate context of the atomic constraint (13.10.3.1 [temp.deduct.general])**, the constraint is not satisfied. Otherwise, the lvalue-to-rvalue conversion (7.3.2 [conv.lval]) is performed if necessary, and E shall be a constant expression of type bool. The constraint is satisfied if and only if evaluation of E results in true.

2. Change in 13.10.3.1 [temp.deduct.general] paragraph 9 as follows:

> ~~A *lambda-expression* appearing in a function type or a template parameter~~ **Substituting into the body of a *lambda-expression*** is ~~not considered part of~~ **never in** the immediate context ~~for the purposes of template argument deduction~~ **of substitution into the *lambda-expression***. [*Note 7:* The intent is to avoid requiring implementations to deal with substitution failure involving arbitrary statements.

Proposed resolution (approved by CWG 2023-11-09):

1. Change in 7.5.7.1 [expr.prim.req.general] paragraph 5 as follows:

> The substitution of template arguments into a *requires-expression* ~~may~~ **can** result in the formation of invalid types or expressions in **the immediate context of** its *requirements* **(13.10.3.1 [temp.deduct.general])** or the violation of the semantic constraints of those *requirements*. In such cases, the *requires-expression* evaluates to false; it does not cause the program to be ill-formed. The substitution and semantic constraint checking proceeds in lexical order and stops when a condition that determines the result of the *requires-expression* is encountered. If substitution (if any) and semantic constraint checking succeed, the *requires-expression* evaluates to true.

2. Change in 13.5.2.3 [temp.constr.atomic] paragraph 3 as follows:

> To determine if an atomic constraint is satisfied, the parameter mapping and template arguments are first substituted into its expression. If substitution results in an invalid type or expression **in the immediate context of the atomic constraint (13.10.3.1 [temp.deduct.general])**, the constraint is not satisfied. Otherwise, the lvalue-to-rvalue conversion (7.3.2 [conv.lval]) is performed if necessary, and E shall be a constant expression of type bool. The constraint is satisfied if and only if evaluation of E results in true.

3. Change in 13.10.3.1 [temp.deduct.general] paragraph 9 as follows:

> ~~A *lambda-expression* appearing in a function type or a template parameter is not considered part of the immediate context for the purposes of template argument deduction~~. **When substituting into a *lambda-expression*, substitution into its body is not in the immediate context.** [*Note 7:* The intent is to avoid requiring implementations to deal with substitution failure involving arbitrary statements.

2725. Overload resolution for non-call of class member access

Section: 7.6.1.5 [expr.ref] **Status:** ready **Submitter:** Richard Smith **Date:** 2023-04-26

Consider:

```
struct A {
  static void f();
  static void f(int);
} x;
void (*p)() = x.f;   // error
```

This is ill-formed as confirmed by issue 61. Various other changes (see issue 2241) have put the following example into the same category:

```
struct B {
  static void f();
} y;
void (*q)() = y.f;   // error
```

If this is the intended outcome (although major implementations disagree), then the rules in 7.6.1.5 [expr.ref] should be clarified accordingly.

Proposed resolution (approved by CWG 2023-06-13):

Change in 7.6.1.5 [expr.ref] bullet 6.3 as follows:

- ...
- If E2 is an overload set, **the expression shall be the (possibly-parenthesized) left-hand operand of a member function call (7.6.1.3 [expr.call]), and** function overload resolution (12.2 [over.match]) is used to select the function to which E2 refers. The type of E1.E2 is the type of E2 and E1.E2 refers to the function referred to by E2.
  - If E2 refers to a static member function, E1.E2 is an lvalue.
  - Otherwise (when E2 refers to a non-static member function), E1.E2 is a prvalue. ~~The expression can be used only as the left-hand operand of a member function call (11.4.2 [class.mfct]).~~ [*Note 5:* Any redundant set of parentheses surrounding the expression is ignored (7.5.3 [expr.prim.paren]). —*end note*]
- ...

This also addresses issue 1038.

2733. Applying `[[maybe_unused]]` to a label

Section: 9.12.8 [dcl.attr.unused] **Status:** ready **Submitter:** Barry Revzin **Date:** 2023-05-25

Subclause 9.12.8 [dcl.attr.unused] paragraph 2 specifies:

> The attribute may be applied to the declaration of a class, a *typedef-name*, a variable (including a structured binding declaration), a non-static data member, a function, an enumeration, or an enumerator.

Absent from that list are labels, but both gcc and clang accept `[[maybe_unused]]` on a label, and behave accordingly.

Proposed resolution (approved by CWG 2023-07-14)

Change in 9.12.8 [dcl.attr.unused] as follows:

> The *attribute-token* `maybe_unused` indicates that a name**, label,** or entity is possibly intentionally unused. No *attribute-argument-clause* shall be present.
>
> The attribute may be applied to the declaration of a class, ~~a~~ *typedef-name*, ~~a~~ variable (including a structured binding declaration), ~~a~~ non-static data member, ~~a~~ function, ~~an~~ enumeration, or ~~an~~ enumerator**, or to an *identifier* label (8.2 [stmt.label])**.

A name or entity declared without the `maybe_unused` attribute can later be redeclared with the attribute and vice versa. An entity is considered marked after the first declaration that marks it.

Recommended practice: For an entity marked `maybe_unused`, implementations should not emit a warning that the entity or its structured bindings (if any) are used or unused. For a structured binding declaration not marked `maybe_unused`, implementations should not emit such a warning unless all of its structured bindings are unused. **For a label to which `maybe_unused` is applied, implementations should not emit a warning that the label is used or unused.**

[*Example 1:*

```
  [[maybe_unused]] void f([[maybe_unused]] bool thing1,
                          [[maybe_unused]] bool thing2) {
    [[maybe_unused]] bool b = thing1 && thing2;
    assert(b);
#ifdef NDEBUG
    goto x;
#endif
    [[maybe_unused]] x:
  }
```

Implementations should not warn that `b` **or `x`** is unused, whether or not `NDEBUG` is defined. — *end example*]

CWG 2023-07-14

CWG has reviewed and approved the proposed resolution. However, this is a new (albeit small) feature, thus forwarding to EWG via paper issue 1585 for approval.

EWG 2023-11-07

Accept the proposed resolution, forward to CWG for inclusion in C++26.

2747. Cannot depend on an already-deleted splice

Section: 5.2 [lex.phases] **Status:** ready **Submitter:** Jim X **Date:** 2021-09-14

(From editorial issue 4903.)

Subclause 5.2 [lex.phases] paragraph 2 specifies:

> ... Each sequence of a backslash character (\) immediately followed by zero or more whitespace characters other than new-line followed by a new-line character is deleted, splicing physical source lines to form logical source lines. ... A source file that is not empty and that does not end in a new-line character, or that ends in a splice, shall be processed as if an additional new-line character were appended to the file.

This is confusing, because the first sentence deletes all splices, and then the last sentence checks for a splice that has already been deleted.

Proposed resolution (approved by CWG 2023-07-14):

Change in 5.2 [lex.phases] paragraph 2 as follows:

> ... Each sequence of a backslash character (\) immediately followed by zero or more whitespace characters other than new-line followed by a new-line character is deleted, splicing physical source lines to form logical source lines. ... A source file that is not empty and that **(after splicing)** does not end in a new-line character~~, or that ends in a splice,~~ shall be processed as if an additional new-line character were appended to the file.

CWG 2023-07-14

CWG noted that a lone backslash at the end of a file remains (in the status quo and with the proposed change) and turns into an ill-formed *preprocessing-token*. The wording as amended seems sufficiently clear to consider issue 1698 resolved.

2749. Treatment of "pointer to void" for relational comparisons

Section: 7.6.9 [expr.rel] **Status:** ready **Submitter:** lprv **Date:** 2023-03-12

(From editorial issue 6173.)

Subclause 7.6.9 [expr.rel] paragraph 4 and paragraph 5 specify:

> The result of comparing unequal pointers to objects [Footnote:] is defined in terms of a partial order consistent with the following rules: ...
>
> [Note 1: A relational operator applied to unequal function pointers or to unequal pointers to void yields an unspecified result. -- end note]

Comparing pointers to objects that are stored in a variable of type "pointer to void" should be fine.

Proposed resolution (approved by CWG 2023-06-16):

Change in 7.6.9 [expr.rel] paragraph 4 and paragraph 5 as follows:

> The result of comparing unequal pointers to objects [Footnote: ...] is defined in terms of a partial order consistent with the following rules: ...
>
> [Note 1: A relational operator applied to unequal function pointers ~~or to unequal pointers to void~~ yields an unspecified result. **A pointer value of type "pointer to *cv* `void`" can point to an object (6.8.4 [basic.compound]).** -- end note]

2753. Storage reuse for string literal objects and backing arrays

Section: 6.7.2 [intro.object] **Status:** ready **Submitter:** Brian Bi **Date:** 2023-06-29

Subclause 6.7.2 [intro.object] paragraph 9 specifies the general principle that two objects with overlapping lifetimes have non-overlapping storage, which can be observed by comparing addresses:

> Unless an object is a bit-field or a subobject of zero size, the address of that object is the address of the first byte it occupies. Two objects with overlapping lifetimes that are not bit-fields may have the same address if one is nested within the other, or if at least one is a subobject of zero size and they are of different types; otherwise, they have distinct addresses and occupy disjoint bytes of storage.

After P2752, there are two exceptions: string literal objects and backing arrays for initializer lists.

Subclause 5.13.5 [lex.string] paragraph 9 specifies:

> Evaluating a *string-literal* results in a string literal object with static storage duration (6.7.5 [basic.stc]). Whether all *string-literal*s are distinct (that is, are stored in nonoverlapping objects) and whether successive evaluations of a *string-literal* yield the same or a different object is unspecified.

Subclause 9.4.4 [dcl.init.ref] paragraph 5, after application of P2752R3 (approved in June, 2023), specifies:

> Whether all backing arrays are distinct (that is, are stored in non-overlapping objects) is unspecified.

It is unclear whether a backing array can overlap with a string literal object.

Furthermore, it is unclear whether any such object can overlap with named objects or temporaries, for example:

```
const char (&r) [] = "foo";
const char a[] = {'f', 'o', 'o', '\0'};

int main() {
  assert(&r == &a);   // allowed not to fail?
}
```

Proposed resolution (approved by CWG 2023-11-09):

1. Add a new paragraph before 6.7.2 [intro.object] paragraph 9 and change the latter as follows:

> An object is a *potentially non-unique object* if it is a string literal object (5.13.5 [lex.string]), the backing array of an initializer list (9.4.4 [dcl.init.ref]), or a subobject thereof.
>
> Unless an object is a bit-field or a subobject of zero size, the address of that object is the address of the first byte it occupies. Two objects with overlapping lifetimes that are not bit-fields may have the same address if one is nested within the other, or if at least one is a subobject of zero size and they are of different types**, or if they are both potentially non-unique objects**; otherwise, they have distinct addresses and occupy disjoint bytes of storage.
>
> [Example 2:
>
> ```
> static const char test1 = 'x';
> static const char test2 = 'x';
> const bool b = &test1 != &test2;  // always true
> ```
>
> ```
> static const char (&r) [] = "x";
> static const char *s = "x";
> static std::initializer_list<char> il = { 'x' };
> const bool b2 = r != il.begin();         // unspecified result
> const bool b3 = r != s;                  // unspecified result
> const bool b4 = il.begin() != &test1;    // always true
> const bool b5 = r != &test1;             // always true
> ```
>
> -- end example]

2. Change in subclause 5.13.5 [lex.string] paragraph 9 as follows:

> Evaluating a *string-literal* results in a string literal object with static storage duration (6.7.5 [basic.stc]). **[Note: String literal objects are potentially non-unique (6.7.2 [intro.object]).** Whether ~~all *string-literals* are distinct (that is, are stored in nonoverlapping objects) and whether~~ successive evaluations of a *string-literal* yield the same or a different object is unspecified. **-- end note]**

3. Change in subclause 9.4.4 [dcl.init.ref] paragraph 5, after application of P2752R3 (approved in June, 2023), as follows:

> ~~Whether all backing arrays are distinct (that is, are stored in non-overlapping objects) is unspecified.~~ **[Note: Backing arrays are potentially non-unique objects (6.7.2 [intro.object]). -- end note]**

CWG 2023-07-14

CWG resolved that a named or temporary object is always disjoint from any other object, and thus cannot overlap with a string literal object or a backing array. The lines `b4` and `b5` in the example highlight that outcome.

Backing arrays and string literals can arbitrarily overlap among themselves; CWG believes the proposed wording achieves that outcome.

The ancillary question how address comparisons between potentially non-unique objects are treated during constant evaluation is handled in issue 2765.

2754. Using *this in explicit object member functions that are coroutines

Section: 9.5.4 [dcl.fct.def.coroutine] **Status:** ready **Submitter:** Christof Meerwald **Date:** 2023-06-23

Subclause 9.5.4 [dcl.fct.def.coroutine] paragraph 4 specifies:

> In the following, pi is an lvalue of type Pi , where p1 denotes the object parameter and pi+1 denotes the ith non-object function parameter for a non-static member function, and pi denotes the ith function parameter otherwise. For a non-static member function, q1 is an lvalue that denotes `*this`; any other qi is an lvalue that denotes the parameter copy corresponding to pi , as described below.

An explicit object member function is a non-static member function, but there is no `this`.

Proposed resolution (approved by CWG 2023-07-14):

Change in 9.5.4 [dcl.fct.def.coroutine] paragraph 4 as follows:

> In the following, pi is an lvalue of type Pi , where p1 denotes the object parameter and pi+1 denotes the ith non-object function parameter for ~~a non-static~~ **an implicit object** member function, and pi denotes the ith function parameter otherwise. For ~~a non-static~~ **an implicit object** member function, q1 is an lvalue that denotes `*this`; any other qi is an lvalue that denotes the parameter copy corresponding to pi, as described below.

2755. Incorrect wording applied by P2738R1

Section: 7.7 [expr.const] **Status:** ready **Submitter:** Jens Maurer **Date:** 2023-06-28

P2738R1 (constexpr cast from void*: towards constexpr type-erasure) applied incorrect wording to 7.7 [expr.const] bullet 5.14:

- ...
- a conversion from a prvalue P of type ”pointer to cv void” to a pointer-to-object type T unless P points to an object whose type is similar to T;
- ...

The issue is that `T` is defined to be a pointer type, but the "similar to" phrasing uses it as the pointee type.

Proposed resolution (approved by CWG 2023-07-14):

Change in 7.7 [expr.const] bullet 5.14 as follows:

- ...
- a conversion from a prvalue P of type ”pointer to cv void” to a ~~pointer-to-object~~ type ~~T~~ **"*cv1* pointer to `T`", where `T` is not *cv2* `void`,** unless P points to an object whose type is similar to T;
- ...

2758. What is "access and ambiguity control"?

Section: 7.6.2.9 [expr.delete] **Status:** ready **Submitter:** CWG **Date:** 2023-06-12

Subclause 7.6.2.9 [expr.delete] paragraph 12 specifies:

> Access and ambiguity control are done for both the deallocation function and the destructor (11.4.7 [class.dtor], 11.4.11 [class.free]).

It is unclear what that means. In particular, ambiguity checking is part of overload resolution, and access checking requires a point of reference.

Proposed resolution (approved by CWG 2023-08-25):

1. Change in 7.6.2.9 [expr.delete] paragraph 6 as follows:

   > If the value of the operand of the *delete-expression* is not a null pointer value and the selected deallocation function (see below) is not a destroying operator delete, **evaluating** the *delete-expression* ~~will invoke~~ **invokes** the destructor (if any) for the object or the elements of the array being deleted. **The destructor shall be accessible from the point where the *delete-expression* appears.** In the case of an array, the elements ~~will be~~ **are** destroyed in order of decreasing address (that is, in reverse order of the completion of their constructor; see 11.9.3 [class.base.init]).

2. Change in 7.6.2.9 [expr.delete] paragraph 10

   > ~~If more than one deallocation function is found, the~~ **The deallocation** function to be called is selected as follows:
   > - ...
   >
   > Unless the deallocation function is selected at the point of definition of the dynamic type's virtual destructor, the selected deallocation function shall be accessible from the point where the *delete-expression* appears.

3. Remove 7.6.2.9 [expr.delete] paragraph 12:

   > ~~Access and ambiguity control are done for both the deallocation function and the destructor (11.4.7 [class.dtor], 11.4.11 [class.free]).~~

2759. [[no_unique_address] and common initial sequence

Section: 11.4.1 [class.mem.general] **Status:** ready **Submitter:** Richard Smith **Date:** 2020-11-10

The interaction of [[no_unique_address]] and the definition of common initial sequence is still problematic. Subclause 11.4.1 [class.mem.general] bullet 23.3 specifies that corresponding members in a common initial sequence are not allowed to differ with respect to the presence or absence of a [[no_unique_address]] attribute. However, the Itanium ABI will not allocate two successive data members of the same empty class type at the same address, causing non-conforming behavior for the following example:

```
struct A {};
struct B {};

struct C {
 [[no_unique_address]] A a;
 [[no_unique_address]] B b;
};

struct D {
 [[no_unique_address]] A a1;
 [[no_unique_address]] A a2;
};

static_assert(offsetof(C, b) == offsetof(D, a2));
```

See Itanium ABI issue 108.

Since "common initial sequence" and "layout compatible" are concepts mostly used for C compatibility, but [[no_unique_address]] does not exist in C, it seems reasonable to terminate a common initial sequence at the first data member that is declared [[no_unique_address]].

Another concern is the behavior of `std::is_layout_compatible` on implementations that ignore [[no_unique_address]]. On such an implementation, the following example would be considered layout-compatible, although it actually is not:

```
struct E {};

struct A {
  E e;
  int i;
};

struct B {
  [[no_unique_address]] E e;
  int i;
};

static_assert(
  std::is_layout_compatible_v<A, B>
);
```

Alternative possible resolution [SUPERSEDED]:

Change in 11.4.1 [class.mem.general] paragraph 23 as follows:

> The common initial sequence of two standard-layout struct (11.2 [class.prop]) types is the longest sequence of non-static data members and bit-fields in declaration order, starting with the first such entity in each of the structs, such that
>
> - corresponding entities have layout-compatible types (6.8 [basic.types]),
> - corresponding entities have the same alignment requirements (6.7.6 [basic.align]),
> - ~~either both entities are declared with the `no_unique_address` attribute (9.12.11 [dcl.attr.nouniqueaddr]) or neither is,~~ **neither entity is declared with the `no_unique_address` attribute (9.12.11 [dcl.attr.nouniqueaddr]),** and
> - either both entities are bit-fields with the same width or neither is a bit-field.

Proposed resolution (approved by CWG 2023-08-25):

Change in 11.4.1 [class.mem.general] paragraph 23 as follows:

> The common initial sequence of two standard-layout struct (11.2 [class.prop]) types is the longest sequence of non-static data members and bit-fields in declaration order, starting with the first such entity in each of the structs, such that
>
> - corresponding entities have layout-compatible types (6.8 [basic.types]),
> - corresponding entities have the same alignment requirements (6.7.6 [basic.align]),
> - ~~either both entities are declared with the `no_unique_address` attribute (9.12.11 [dcl.attr.nouniqueaddr]) or neither is,~~ **if a *has-attribute-expression* (15.2 [cpp.cond]) is not 0 for the `no_unique_address` attribute, then neither entity is declared with the `no_unique_address` attribute (9.12.11 [dcl.attr.nouniqueaddr]),** and
> - either both entities are bit-fields with the same width or neither is a bit-field.

2760. Defaulted constructor that is an immediate function

Section: 7.7 [expr.const] **Status:** ready **Submitter:** Corentin Jabot **Date:** 2023-07-08

Consider:

```
consteval int f(int);
struct S {
 int x = f(0);
 S() = default;
};

int main() {
  S s;     // OK?
}
```

Is `S` an immediate function?

The relevant specification is in 7.7 [expr.const] paragraph 18:

> An immediate function is a function or constructor that is
>
> - declared with the `consteval` specifier, or
> - an immediate-escalating function F whose function body contains an immediate-escalating expression E such that E's innermost enclosing non-block scope is F's function parameter scope.

Suggested resolution [SUPERSEDED]:

Change in 7.7 [expr.const] paragraph 18 as follows:

> An immediate function is a function or constructor that is
>
> - declared with the `consteval` specifier, or
> - an immediate-escalating function F whose function body contains an immediate-escalating expression E such that E's innermost enclosing non-block scope is F's function parameter scope**, or**
> - an immediate-escalating default constructor of a class which has at least one non-static data member with an immediate-escalating default member initializer.

Proposed resolution (approved by CWG 2023-08-25):

1. Change in 7.7 [expr.const] paragraph 18 as follows:

   > An immediate function is a function or constructor that is
   > - declared with the `consteval` specifier, or
   > - an immediate-escalating function F whose function body contains an immediate-escalating expression E such that E's innermost enclosing non-block scope is F's function parameter scope. **[Note: Default member initializers used to initialize a base or member subobject (11.9.3 [class.base.init]) are considered to be part of the function body (9.5.1 [dcl.fct.def.general]). -- end note]**

2. Change in 9.5.1 [dcl.fct.def.general] paragraph 1 as follows:

   > Any informal reference to the body of a function should be interpreted as a reference to the non-terminal *function-body***, including, for a constructor, default member initializers or default initialization used to initialize a base or member subobject in the absence of a *mem-initializer-id* (11.9.3 [class.base.init])**.

2761. Implicitly invoking the deleted destructor of an anonymous union member

Section: 11.4.7 [class.dtor] **Status:** ready **Submitter:** Corentin Jabot **Date:** 2023-07-11

Consider:

```
struct S{
  ~S() {}
};

struct A {
  union {
    S arr_;
  };
  ~A(); // user-provided!
};

auto foo() {
  return A{S()};
}
```

Does the destructor of `A` attempt to destroy the (unnamed) data member that is the anonymous union? The latter has a deleted destructor per 11.4.7 [class.dtor]. For the default constructor, 11.9.3 [class.base.init] paragraph 9.2 prevents the corresponding construction.

Proposed resolution (approved by CWG 2023-08-25):

1. Change in 9.4.2 [dcl.init.aggr] paragraph 9 as follows:

   > The destructor for each element of class type **other than an anonymous union member** is potentially invoked (11.4.7 [class.dtor]) from the context where the aggregate initialization occurs

2. Change in 11.4.7 [class.dtor] paragraph 13 as follows:

After executing the body of the destructor and destroying any objects with automatic storage duration allocated within the body, a destructor for class X calls the destructors for X's direct non-variant non-static data members **other than anonymous unions**, the destructors for X's non-virtual direct base classes and, if X is the most derived class (11.9.3 [class.base.init]), its destructor calls the destructors for X's virtual base classes. All destructors are called as if they were referenced with a qualified name, that is, ignoring any possible virtual overriding destructors in more derived classes. Bases and members are destroyed in the reverse order of the completion of their constructor (see 11.9.3 [class.base.init]).

3. Change in 14.3 [except.ctor] paragraph 3 as follows:

A subobject is *known to be initialized* if **it is not an anonymous union member and** its initialization is specified

- in 11.9.3 [class.base.init] for initialization by constructor,
- in 11.4.5.3 [class.copy.ctor] for initialization by defaulted copy/move constructor,
- in 11.9.4 [class.inhctor.init] for initialization by inherited constructor,
- in 9.4.2 [dcl.init.aggr] for aggregate initialization,
- in 7.5.5.3 [expr.prim.lambda.capture] for the initialization of the closure object when evaluating a lambda-expression,
- in 9.4.1 [dcl.init.general] for default-initialization, value-initialization, or direct-initialization of an array.

2762. Type of implicit object parameter

Section: 12.2.2.1 [over.match.funcs.general] **Status:** ready **Submitter:** Jim X **Date:** 2023-07-11

Subclause 12.2.2.1 [over.match.funcs.general] paragraph 4 specifies:

For implicit object member functions, the type of the implicit object parameter is

- "lvalue reference to cv X" for functions declared without a *ref-qualifier* or with the & *ref-qualifier*
- "rvalue reference to cv X" for functions declared with the && *ref-qualifier*

where X is the class of which the function is a member and cv is the cv-qualification on the member function declaration.

Since a member of some class C is also a member of any class derived from C, this specification is unclear.

Proposed resolution (approved by CWG 2023-08-25):

Change in 12.2.2.1 [over.match.funcs.general] paragraph 4 as follows:

For implicit object member functions, the type of the implicit object parameter is

- "lvalue reference to cv X" for functions declared without a *ref-qualifier* or with the & *ref-qualifier*
- "rvalue reference to cv X" for functions declared with the && *ref-qualifier*

where X is the class of which the function is a **direct** member and cv is the cv-qualification on the member function declaration.

2763. Ignorability of [[noreturn]] during constant evaluation

Section: 7.7 [expr.const] **Status:** ready **Submitter:** Jiang An **Date:** 2023-07-10

Subclause 9.12.10 [dcl.attr.noreturn] paragraph 2 specifies:

If a function f is called where f was previously declared with the `noreturn` attribute and f eventually returns, the behavior is undefined.

Undefineed behavior is, in general, detected during constant evaluation, thus requiring an implementation to actually support the `noreturn` attribute, such as in the following example:

```
[[noreturn]] constexpr void f() {}
constexpr int x = (f(), 0);
```

It might be desirable to treat the `assume` and `noreturn` attributes alike in that regard.

Proposed resolution (approved by CWG 2023-07-14) [SUPERSEDED]:

Split into a separate paragraph and change 7.7 [expr.const] paragraph 5 as follows:

It is unspecified whether E is a core constant expression if E satisfies the constraints of a core constant expression, but evaluation of E would evaluate

- an operation that has undefined behavior as specified in Clause 16 through Clause 33,
- an invocation of the va_start macro (17.13.2 [cstdarg.syn]), ~~or~~
- a `return` statement in a function that was previously declared with the `noreturn` attribute (9.12.10 [dcl.attr.noreturn]), or
- a statement with an assumption (9.12.3 [dcl.attr.assume]) whose converted *conditional-expression*, if evaluated where the assumption appears, would not disqualify E from being a core constant expression and would not evaluate to `true`. [Note: ....]

CWG 2023-07-14

As an alternative, all of 9.12 [dcl.attr] could be added to the "library undefined behavior" bullet. However, CWG felt that a case-by-case consideration is warranted, given that assumptions set precedent in requiring special treatment.

Possible resolution (reviewed by CWG 2023-08-25) [SUPERSEDED]:

Split into a separate paragraph and change 7.7 [expr.const] paragraph 5 as follows:

- ...
- an operation that would have undefined behavior as specified in Clause 4 [intro] through Clause 15 [cpp], excluding 9.12.3 [dcl.attr.assume] **and 9.12.10 [dcl.attr.noreturn]**; [Footnote: ...]
- ...

It is unspecified whether E is a core constant expression if E satisfies the constraints of a core constant expression, but evaluation of E would evaluate

- an operation that has undefined behavior as specified in Clause 16 through Clause 33,
- an invocation of the va_start macro (17.13.2 [cstdarg.syn]), ~~or~~
- a `return` statement in a function that was previously declared with the `noreturn` attribute (9.12.10 [dcl.attr.noreturn]), or
- a statement with an assumption (9.12.3 [dcl.attr.assume]) whose converted *conditional-expression*, if evaluated where the assumption appears, would not disqualify E from being a core constant expression and would not evaluate to `true`. [Note: ....]

Proposed resolution (approved by CWG 2023-11-09):

Split into a separate paragraph and change 7.7 [expr.const] paragraph 5 as follows:

- ...
- an operation that would have undefined behavior as specified in Clause 4 [intro] through Clause 15 [cpp], excluding 9.12.3 [dcl.attr.assume] **and 9.12.10 [dcl.attr.noreturn]**; [Footnote: ...]
- ...

It is unspecified whether E is a core constant expression if E satisfies the constraints of a core constant expression, but evaluation of E would evaluate

- an operation that has undefined behavior as specified in Clause 16 through Clause 33,
- an invocation of the va_start macro (17.13.2 [cstdarg.syn]), ~~or~~
- a call to a function that was previously declared with the `noreturn` attribute (9.12.10 [dcl.attr.noreturn]) and that call returns to its caller, or
- a statement with an assumption (9.12.3 [dcl.attr.assume]) whose converted *conditional-expression*, if evaluated where the assumption appears, would not disqualify E from being a core constant expression and would not evaluate to `true`. [Note: ....]

2764. Use of placeholders affecting name mangling

Section: 6.4.1 [basic.scope.scope] **Status:** ready **Submitter:** Hubert Tong **Date:** 2023-07-05

Paper P2169R4 (A nice placeholder with no name), as approved by WG21 in Varna, added a placeholder facility. The intent was that the use of placeholders is sufficiently limited such that they never need to be mangled. Quote from 6.4.1 [basic.scope.scope] paragraph 5 as modified by the paper:

> A declaration is name-independent if its name is _ and it declares a variable with automatic storage duration, a structured binding not inhabiting a namespace scope, the variable introduced by an *init-capture*, or a non-static data member.

The following example does not seem to follow that intent:

```
struct A { A(); };
inline void f() {
  static union { A _{}; };
  static union { A _{}; };
}
void g() { return f(); }
```

The preceding example needs handling similar to the following example, which is unrelated to the placeholder feature:

```
struct A { A(); };
inline void f() {
  { static union { A a{}; }; }
  { static union { A a{}; }; }
}
void g() { return f(); }
```

A similar problem may arise for `static` or `thread_local` structured bindings at block scope.

Finally, another example involving placeholders in anonymous unions:

```
static union { int _ = 42; };
int &ref = _;
int foo() { return 13; }
static union { int _ = foo(); };
int main() { return ref; }
```

Possible resolution (reviewed by CWG 2023-08-25) [SUPERSEDED]:

Change in 6.4.1 [basic.scope.scope] paragraph 5 and add bullets as follows:

> **A class is *name-dependent* if it is an anonymous union declared at namespace scope or with a *storage-class-specifier* (11.5.2 [class.union.anon]).** A declaration is *name-independent* if its name is _ and it declares
>
> - a variable with automatic storage duration,
> - a structured binding **with no *storage-class-specifier* and** not inhabiting a namespace scope,
> - the variable introduced by an *init-capture*, or
> - a non-static data member **of other than a name-dependent class**.

Proposed resolution (approved by CWG 2023-09-15):

Change in 6.4.1 [basic.scope.scope] paragraph 5 and add bullets as follows:

> A declaration is *name-independent* if its name is _ and it declares
>
> - a variable with automatic storage duration,
> - a structured binding **with no *storage-class-specifier* and** not inhabiting a namespace scope,
> - the variable introduced by an *init-capture*, or
> - a non-static data member **of other than an anonymous union**.

2768. Assignment to enumeration variable with a *braced-init-list*

Section: 7.6.19 [expr.ass] **Status:** ready **Submitter:** Shafik Yaghmour **Date:** 2023-07-06

Consider:

```
enum class E {E1};

void f() {
  E e;
  e = E{0}; // #1
  e = {0};  // #2
}
```

#1 first initializes a temporary of type `E` and then assigns that to `e`. For #2, 7.6.19 [expr.ass] bullet 8.1 specifies that #2 is equivalent to #1:

> A *braced-init-list* may appear on the right-hand side of
>
> - an assignment to a scalar, in which case the initializer list shall have at most a single element. The meaning of `x = {v}`, where T is the scalar type of the expression x, is that of `x = T{v}`. The meaning of `x = {}` is `x = T{}`.
> - ...

However, there is no syntactic hint that #2 would invoke direct-initialization, and in fact gcc, icc, and MSVC reject #2, but clang accepts.

Proposed resolution (approved by CWG 2023-11-06):

Change in 7.6.19 [expr.ass] paragraph 8 as follows:

> A *braced-init-list* ***B*** may appear on the right-hand side of
>
> - an assignment to a scalar **of type `T`**, in which case ~~the initializer list~~ ***B*** shall have at most a single element. The meaning of `x = {v}`***B*** **is `x = t`, where `t` is an invented temporary variable declared and initialized as `T t =` *B***~~, where T is the scalar type of the expression x, is that of `x = T{v}`. The meaning of `x = {}` is `x = T{}`~~.
> - an assignment to an object of class type, in which case ~~the initializer list~~ ***B*** is passed as the argument to the assignment operator function selected by overload resolution (12.4.3.2 [over.ass], 12.2 [over.match]).

2772. Missing Annex C entry for linkage effects of *linkage-specification*

Section: C.6.4 [diff.cpp03.dcl.dcl] **Status:** ready **Submitter:** Hubert Tong **Date:** 2023-07-15

With C++11, anonymous namespaces changed from external linkage (with a unique namespace name) to internal linkage. That implies that `extern "C"`, which affects names with external linkage only, no longer has an effect inside anonymous namespaces.

However, a corresponding Annex C entry is missing.

Proposed resolution (approved by CWG 2023-09-15):

Add a new paragraph in C.6.4 [diff.cpp03.dcl.dcl] as follows:

> Affected subclause: 9.11 [dcl.link]
> Change: Names declared in an anonymous namespace changed from external linkage to internal linkage; language linkage applies to names with external linkage only.
> Rationale: Alignment with user expectations.
> Effect on original feature: Valid C++ 2003 code may violate the one-definition rule (6.3 [basic.def.odr]) in this revision of C++. For example:
>
> ```
> namespace { extern "C" { extern int x; } }  // #1, previously external linkage and C language linkage, now internal linkage and C++ language linkage
> namespace A { extern "C" int x = 42; }      // #2, external linkage and C language linkage
> int main(void) { return x; }
> ```
>
> This code is valid in C++ 2003, but #2 is not a definition for #1 in this revision of C++, violating the one-definition rule.

2780. `reinterpret_cast` to reference to function types

Section: 7.6.1.10 [expr.reinterpret.cast] **Status:** ready **Submitter:** Lauri Vasama **Date:** 2023-08-07

Subclause 7.6.1.10 [expr.reinterpret.cast] paragraph 11 specifies:

> A glvalue of type T1, designating an object x, can be cast to the type “reference to T2” if an expression of type “pointer to T1” can be explicitly converted to the type “pointer to T2” using a reinterpret_cast. The result is that of *reinterpret_cast<T2 *>(p) where p is a pointer to x of type “pointer to T1”. No temporary is created, no copy is made, and no constructors (11.4.5 [class.ctor]) or conversion functions (11.4.8 [class.conv]) are called. [Footnote: ...]

The wording does not cover references to function type, only references to object types. All major implementations accept the following example:

```
void f() {}

void(&g())(int) {
  return reinterpret_cast<void(&)(int)>(f);
}
```

Proposed resolution (approved by CWG 2023-09-15):

Change in 7.6.1.10 [expr.reinterpret.cast] paragraph 11 as follows:

> A glvalue of type T1, designating an object **or function** x, can be cast to the type “reference to T2” if an expression of type “pointer to T1” can be explicitly converted to the type “pointer to T2” using a `reinterpret_cast`. The result is that of `*reinterpret_cast<T2 *>(p)` where `p` is a pointer to `x` of type “pointer to T1”. No temporary is created, no copy is made, and no constructors (11.4.5 [class.ctor]) or conversion functions (11.4.8 [class.conv]) are called. [Footnote: ...]

2783. Handling of deduction guides in *global-module-fragment*

Section: 10.4 [module.global.frag] **Status:** ready **Submitter:** Daniela Engert **Date:** 2023-08-21

Consider:

```
// header "S.h"

template<class T>
struct S {
  S(const T*);
};
template<class T>
S(T*) -> S<T>

// translation unit
module;
#include "S.h"

export module M;
export using ::S;
```

Obviously, the *using-declaration* referring to the class template `S` is exported by `M`, but what about the deduction guide of `S`?

Proposed resolution (approved by CWG 2023-08-25) [SUPERSEDED]:

Add a new bullet after 10.4 [module.global.frag] bullet 3.5.7 as follows:

- ...
- there exists a declaration M that is not a *namespace-definition* for which M is decl-reachable from S and either
  - ...
  - one of M and D declares a template and the other declares a partial or explicit specialization or an implicit or explicit instantiation of that template, or
  - one of M and D declares a class template and the other declares a deduction guide for that template, or

Proposed resolution (approved by CWG 2023-10-06):

Add a new bullet after 10.4 [module.global.frag] bullet 3.5.7 as follows:

- ...
- there exists a declaration M that is not a *namespace-definition* for which M is decl-reachable from S and either
  - ...
  - one of M and D declares a template and the other declares a partial or explicit specialization or an implicit or explicit instantiation of that template, or
  - M declares a class template and D is a deduction guide for that template, or

---

2785. Type-dependence of *requires-expression*

Section: 13.8.3.3 [temp.dep.expr] **Status:** ready **Submitter:** CWG **Date:** 2023-07-17

(Split off from issue 2774.)

Subclause 13.8.3.3 [temp.dep.expr] is lacking specifiation about the type-dependence of *requires-expression*s.

Proposed resolution (approved by CWG 2023-08-25):

Change in 13.8.3.3 [temp.dep.expr] paragraph 4 as follows:

> Expressions of the following forms are never type-dependent (because the type of the expression cannot be dependent):
>
> ```
> ...
> noexcept ( expression )
> requires-expression
> ```

2789. Overload resolution with implicit and explicit object member functions

Section: 12.2.4.1 [over.match.best.general] **Status:** ready **Submitter:** Corentin Jabot **Date:** 2023-08-08

Consider:

```
template <typename T = int>
struct S {
  constexpr void f();                      // #1
  constexpr void f(this S&) requires true; // #2
};

void test() {
  S<> s;
  s.f();                 // #3
}
```

With the current rules, the call at #3 is ambiguous, even though #2 is more constrainted.

Proposed resolution (approved by CWG 2023-11-07):

Change in 12.2.4.1 [over.match.best.general] bullet 2.6 as follows:

- ...
- F1 and F2 are non-template functions ~~with~~ **and**
  - **they have** the same ~~parameter-type-lists~~ **non-object-parameter-type-lists (9.3.4.6 [dcl.fct]),** and
  - if they are member functions, both are direct members of the same class, and
  - if both are non-static member functions, they have the same types for their object parameters, and
  - F1 is more constrained than F2 according to the partial ordering of constraints described in 13.5.5 [temp.constr.order],

  or if not that,

  [Example:

  ```
  template <typename T = int>
  struct S {
    constexpr void f();                      // #1
    constexpr void f(this S&) requires true; // #2
  };

  void test() {
    S<> s;
    s.f();                 // calls #2
  }
  ```

  -- end example]

- ...

2791. Unclear phrasing about "returning to the caller"

Section: 8.7.4 [stmt.return] **Status:** ready **Submitter:** Jan Schultke **Date:** 2023-08-23

In 8.7.4 [stmt.return] and 8.7.5 [stmt.return.coroutine], the standard uses the phrasing "returns to its caller" when specifying `return` or `co_return`. It would be better to talk about transfer of control, which is a term used elsewhere in the standard.

Proposed resolution (approved by CWG 2023-10-06):

1. Change in 7.6.2.4 [expr.await] paragraph 1 as follows:

   > The co_await expression is used to suspend evaluation of a coroutine (9.5.4 [dcl.fct.def.coroutine]) while awaiting completion of the computation represented by the operand expression. **Suspending the evaluation of a coroutine transfers control to its caller or resumer.**

2. Change 8.7.4 [stmt.return] paragraph 1 as follows:

   > A function returns **control** to its caller by the return statement.

3. Change 8.7.5 [stmt.return.coroutine] paragraph 1 as follows:

~~A coroutine returns to its caller or resumer (9.5.4 [dcl.fct.def.coroutine]) by the `co_return` statement or when suspended (7.6.2.4 [expr.await]).~~ **A `co_return` statement transfers control to the caller or resumer of a coroutine (9.5.4 [dcl.fct.def.coroutine]).** A coroutine shall not enclose a `return` statement (8.7.4 [stmt.return]).

4. Change in 9.5.4 [dcl.fct.def.coroutine] paragraph 10 as follows:

> If the allocation function returns nullptr, the coroutine ~~returns~~ **transfers** control to the caller of the coroutine and the return value is obtained by a call to `T::get_return_object_on_allocation_failure()`, where T is the promise type.

2792. Clean up specification of `noexcept` operator

Section: 7.6.2.7 [expr.unary.noexcept] **Status:** ready **Submitter:** Jan Schultke **Date:** 2023-08-30

The introductory sentence "can throw an exception" is misleading, because it might be interpreted to cover exceptions thrown as the result of encountering undefined behavior.

Proposed resolution (approved by CWG 2023-10-06):

Change all of 7.6.2.7 [expr.unary.noexcept] as follows:

> ~~The noexcept operator determines whether the evaluation of its operand, which is an unevaluated operand (7.2.3 [expr.context]), can throw an exception (14.2 [except.throw]).~~
>
> *noexcept-expression*:
> `noexcept (` *expression* `)`
>
> The operand of the `noexcept` operator is an unevaluated operand (7.2.3 [expr.context]). If the operand is a prvalue, the temporary materialization conversion (7.3.5 [conv.rval]) is applied.
>
> The result of the `noexcept` operator is a prvalue of type `bool`. **The result is `false` if the full-expression of the operand is potentially-throwing (14.5 [except.spec]), and `true` otherwise.**
>
> [*Note 1:* A *noexcept-expression* is an integral constant expression (7.7 [expr.const]). *—end note*]
>
> ~~If the operand is a prvalue, the temporary materialization conversion (7.3.5 [conv.rval]) is applied. The result of the noexcept operator is true unless the full-expression of the operand is potentially-throwing (14.5 [except.spec]).~~

2793. Block-scope declaration conflicting with parameter name

Section: 6.4.3 [basic.scope.block] **Status:** ready **Submitter:** Jason Merrill **Date:** 2023-08-31

Consider:

```
void f(int i) { extern int i; }
```

According to 6.4.3 [basic.scope.block] paragraph 2, the target scope of the declaration is relevant (which would be the global scope), but not the scope in which the name is bound. That seems wrong. For comparison, template parameter names use the latter rule (13.8.2 [temp.local] paragraph 6).

Proposed resolution (approved by CWG 2023-09-15):

> If a declaration that is not a name-independent declaration and ~~whose target scope is~~ **that binds a name in** the block scope S of a
>
> - *compound-statement* of a *lambda-expression*, *function-body*, or *function-try-block*,
> - substatement of a selection or iteration statement that is not itself a selection or iteration statement, or
> - handler of a *function-try-block*
>
> potentially conflicts with a declaration whose target scope is the parent scope of S, the program is ill-formed.

2795. Overlapping empty subobjects with different cv-qualification

Section: 6.7.2 [intro.object] **Status:** ready **Submitter:** Jonathan Caves **Date:** 2023-09-04

Subclause 6.7.2 [intro.object] paragraph 9 specifies:

> ... Two objects with overlapping lifetimes that are not bit-fields may have the same address if one is nested within the other, or if at least one is a subobject of zero size and they are of different types; otherwise, they have distinct addresses and occupy disjoint bytes of storage. [Footnote: ...]

Types `T` and `const T` are different types, but it is unlikely the rule is intending to differentiate along that line.

Suggested resolution [SUPERSEDED]:

Change in 6.7.2 [intro.object] paragraph 9 as follows:

... Two objects with overlapping lifetimes that are not bit-fields may have the same address if one is nested within the other, or if at least one is a subobject of zero size and they are of different types **(ignoring top-level cv-qualifiers)**; otherwise, they have distinct addresses and occupy disjoint bytes of storage. [Footnote: ...]

Proposed resolution (approved by CWG 2023-09-15):

(Hypothetically, pointer-to-member types can be empty, but might differ in non-top-level cv-qualification.)

Change in 6.7.2 [intro.object] paragraph 9 as follows:

... Two objects with overlapping lifetimes that are not bit-fields may have the same address if one is nested within the other, or if at least one is a subobject of zero size and they are **not** of ~~different~~ **similar** types **(7.3.6 [conv.qual])**; otherwise, they have distinct addresses and occupy disjoint bytes of storage. [Footnote: ...]

2796. Function pointer conversions for relational operators

Section: 7.6.9 [expr.rel] **Status:** ready **Submitter:** Alisdair Meredith **Date:** 2023-09-14

Consider:

```
void f() {}
void g() noexcept {}

void q() {
  bool b1 = f == g;      // OK
  bool b2 = f > g;       // error: different types
}
```

For the equality operators, 7.6.10 [expr.eq] paragraph 3 specifies:

If at least one of the operands is a pointer, pointer conversions (7.3.12 [conv.ptr]), function pointer conversions (7.3.14 [conv.fctptr]), and qualification conversions (7.3.6 [conv.qual]) are performed on both operands to bring them to their composite pointer type (7.2.2 [expr.type]). Comparing pointers is defined as follows: ...

In contrast, the corresponding rule for relational operators in 7.6.9 [expr.rel] paragraph 3 specifies:

The usual arithmetic conversions (7.4 [expr.arith.conv]) are performed on operands of arithmetic or enumeration type. If both operands are pointers, pointer conversions (7.3.12 [conv.ptr]) and qualification conversions (7.3.6 [conv.qual]) are performed to bring them to their composite pointer type (7.2.2 [expr.type]). After conversions, the operands shall have the same type.

However, all major implementations accept the example.

Proposed resolution (approved by CWG 2023-10-06):

Change in 7.6.9 [expr.rel] paragraph 3 as follows:

The usual arithmetic conversions (7.4 [expr.arith.conv]) are performed on operands of arithmetic or enumeration type. If both operands are pointers, pointer conversions (7.3.12 [conv.ptr])**, function pointer conversions (7.3.14 [conv.fctptr]),** and qualification conversions (7.3.6 [conv.qual]) are performed to bring them to their composite pointer type (7.2.2 [expr.type]). After conversions, the operands shall have the same type.

2798. Manifestly constant evaluation of the `static_assert` message

Section: 7.7 [expr.const] **Status:** ready **Submitter:** Jason Merrill **Date:** 2023-09-12

The message of a `static_assert` declaration is a *conditional-expression* and thus is not manifestly constant evaluated. Consider this example:

```
struct X {
  std::string s;
  const char *p;
};
consteval X f() { return {.s = "some long string that requires a heap allocation", .p = "hello"}; }

static_assert(cond, f().p);
```

The example is ill-formed, because the immediate invocation `f()` lets a pointer to the heap escape.

Proposed resolution (approved by CWG 2023-10-06):

1. Change in 7.7 [expr.const] paragraph 19 as follows:

   [*Note 11:* **Except for a *static_assert-message*, a** ~~A~~ manifestly constant-evaluated expression is evaluated even in an unevaluated operand (7.2.3 [expr.context]). —*end note*]

2. Change the grammar in 9.1 [dcl.pre] as follows:

   ```
   static_assert-message:
     unevaluated-string
     conditional-expression  constant-expression
   ```

3. Change in 9.1 [dcl.pre] bullet 11.2 as follows:

    - ...
    - if the *static_assert-message* is a ~~*conditional-expression*~~ ***constant-expression*** M, ...

2801. Reference binding with reference-related types

Section: 9.4.4 [dcl.init.ref] **Status:** ready **Submitter:** Brian Bi **Date:** 2023-09-18

Consider:

```
int* p;
const int*&& r = static_cast<int*&&>(p);
```

The intent of core issues 2018 and 2352 was to make this example ill-formed, because it surprisingly introduces a temporary.

Proposed resolution (approved by CWG 2023-10-20):

Change in 9.4.4 [dcl.init.ref] bullet 5.4 as follows:

- Otherwise~~:~~**, `T1` shall not be reference-related to `T2`.**
    - If T1 or T2 is a class type ~~and T1 is not reference-related to T2~~, user-defined conversions are considered using the rules for copy-initialization of an object of type "cv1 T1" by user-defined conversion (9.4 [dcl.init], 12.2.2.5 [over.match.copy], 12.2.2.6 [over.match.conv]); the program is ill-formed if the corresponding non-reference copy-initialization would be ill-formed. The result of the call to the conversion function, as described for the non-reference copy-initialization, is then used to direct-initialize the reference. For this direct-initialization, user-defined conversions are not considered.
    - Otherwise, the initializer expression is implicitly converted to a prvalue of type "T1". The temporary materialization conversion is applied, considering the type of the prvalue to be "cv1 T1", and the reference is bound to the result.

  ~~If T1 is reference-related to T2:~~
    - ~~cv1 shall be the same cv-qualification as, or greater cv-qualification than, cv2; and~~
    - ~~if the reference is an rvalue reference, the initializer expression shall not be an lvalue. [*Note 3:* This can be affected by whether the initializer expression is move-eligible (7.5.4.2 [expr.prim.id.unqual]). —*end note*]~~

2806. Make a *type-requirement* a type-only context

Section: 13.8.1 [temp.res.general] **Status:** ready **Submitter:** Barry Revzin **Date:** 2023-10-10

Consider:

```
template <typename T>
concept C = requires {
  typename T::type<void>;   // template required?
};
```

There is implementation divergence: gcc accepts, clang and MSVC reject.

A *type-requirement* ought to be a type-only context.

Proposed resolution (approved by CWG 2023-10-20):

1. Change in 7.5.7.3 [expr.prim.req.type] paragraph 1 as follows:

    A *type-requirement* asserts the validity of a type. **The component names of a *type-requirement* are those of its *nested-name-specifier* (if any) and *type-name*.** [*Note 1:* The enclosing *requires-expression* will evaluate to `false` if substitution of template arguments fails. —*end note*]

2. Change in 13.8.1 [temp.res.general] paragraph 4 as follows:

    A qualified or unqualified name is said to be in a *type-only context* if it is the terminal name of
    - a *typename-specifier*, ***type-requirement,*** *nested-name-specifier*, *elaborated-type-specifier*, *class-or-decltype*, or
    - ...

2807. Destructors declared `consteval`

Section: 11.4.7 [class.dtor] **Status:** ready **Submitter:** Corentin Jabot **Date:** 2023-09-07

There is a conflict between 9.2.6 [dcl.constexpr] paragraph 2

> A destructor, an allocation function, or a deallocation function shall not be declared with the `consteval` specifier.

and 11.4.7 [class.dtor] paragraph 1

> Each *decl-specifier* of the *decl-specifier-seq* of a prospective destructor declaration (if any) shall be friend, inline, virtual, constexpr, or consteval.

Proposed resolution (approved by CWG 2023-10-20):

Change in 11.4.7 [class.dtor] paragraph 1 as follows:

> Each *decl-specifier* of the *decl-specifier-seq* of a prospective destructor declaration (if any) shall be `friend`, `inline`, `virtual`, **or** `constexpr`~~, or `consteval`~~.

2823. Implicit undefined behavior when dereferencing pointers

Section: 7.6.2.2 [expr.unary.op] **Status:** ready **Submitter:** CWG **Date:** 2023-11-06

Subclause 7.6.2.2 [expr.unary.op] paragraph 1 specifies:

> The unary * operator performs indirection. Its operand shall be a prvalue of type “pointer to T”, where T is an object or function type. The operator yields an lvalue of type T denoting the object or function to which the operand points.

It is unclear what happens if the operand does not point to an object or function.

Proposed resolution (approved by CWG 2023-11-08):

Change in 7.6.2.2 [expr.unary.op] paragraph 1 as follows:

> The unary * operator performs indirection. Its operand shall be a prvalue of type “pointer to T”, where T is an object or function type. The operator yields an lvalue of type T ~~denoting the object or function to which the operand points~~. **If the operand points to an object or function, the result denotes that object or function; otherwise, the behavior is undefined except as specified in 7.6.1.8 [expr.typeid].**